



Java modifier

▼ Type	 Lecture
📅 Date	@January 8, 2022
☰ Lecture #	6
🔗 Lecture URL	https://youtu.be/IO-K87_QXGs
🔗 Notion URL	https://21f1003586.notion.site/Java-modifier-fab02a734e3b4208b6c190821c5e9a79
# Week #	3

Modifiers in Java

- Java uses many modifiers in declarations, to cover different features of object-oriented programming
- `public` vs `private` to support encapsulation of data
- `static` , for entities defined inside classes that exist without creating objects of the class
- `final` , for values that cannot be changed
- These modifiers can be applied to classes, instance variables and methods
- Let’s look at some examples of situations where different combinations make sense

`public` VS `private`

- Faithful implementation of encapsulation necessitates modifiers `public` and `private`
 - Typically, instance variables are `private`
 - Methods to query (accessor) and update (mutator) the state are `public`
- We also use `private` methods sometimes to do some work that shouldn’t be directly accessible to public
- Example: a `Stack` class
 - Data stored in a private array
 - Public methods to push, pop, query if empty

```
public class Stack {
    private int[] values;
    private int size;
    private int top;

    public void push(int i) {
        ...
    }
}
```

```

}

public int pop() {
    ...
}

public boolean isEmpty() {
    return top == 0;
}
}

```

- `push()` needs to check if the stack has space

```

public class Stack {
    ...
    public void push(int i) {
        if(top < size) {
            values[top] = i;
            top += 1;
        } else {
            // Deal with stack overflow
        }
        ...
    }
    ...
}

```

- Deal gracefully with stack overflow
 - `private` methods invoked from within `push()` to check if stack is full and expand storage

```

public class Stack {
    ...
    public void push(int i) {
        if(stack_full()) {
            extend_stack();
        }
        ... // Usual push operation
    }
    ...
    private boolean stack_full() {
        return top == size;
    }
    private void extend_stack() {
        /* Allocate additional space, reset size etc. */
    }
}

```

Accessor and Mutator

- Public methods to query and update private instance variables
- `Date` class
 - Private instance variables `day, month, year`
 - One public accessor/mutator method per instance variable
- Inconsistent updates are now possible
 - Separately set invalid combinations of `day` and `month`

```

public class Date {
    private int day, month, year;

    public int getDay() { ... }
    public int getMonth() { ... }
    public int getYear() { ... }

    public void setDay(int d) { ... }
    public void setMonth(int m) { ... }
    public void setYear(int y) { ... }
}

```

- Instead, only allow combined update

```
public class Date {
    private int day, month, year;

    public int getDay() { ... }
    public int getMonth() { ... }
    public int getYear() { ... }

    public void setDate(int d, int m, int y) {
        ...
        // Validate d-m-y combinations
    }
}
```

static components

- Use `static` for components that exist without creating objects
 - Library functions, `main(), ...`
 - Useful constants like `Math.PI, Integer.MAX_VALUE`
- These `static` components are also `public`
- We do use `private static` sometimes
- Internal constants for bookkeeping
 - Constructor sets unique id for each order

```
public class Order {
    private static int lastorderid = 0;
    private int orderid;
    ...
    public Order(...) {
        lastorderid++;
        orderid = lastorderid;
    }
}
```

- `lastorderid` is private static field
- Common to all objects in the class
- Concurrent updates need some care

final components

- `final` denotes that a value cannot be updated
- Usually used for constants (`public` and `static` instance variables)
 - `Math.PI, Integer.MAX_VALUE`
- What would `final` mean for a method?
 - Cannot re-define functions at run-time

Summary

- `private` and `public` are natural artefacts of encapsulation
 - Usually, instance variables are `private` and methods are `public`
 - However, `private` methods also make sense
- Modifiers `static` and `final` are orthogonal to `public/private`
- Use `private static` instance variables to maintain bookkeeping information across objects in a class
 - Global serial number, count number of objects created, profile method invocations ...
- Usually `final` is used with instance variables to denote constants
- A `final` method cannot be overridden by a subclass
- We can also have `private` classes